Microcontroller Programming

Federico Gabrielli

April 29, 2025

1 Code with simple instructions

The original Arduino code likely includes the `map()` function to convert the potentiometer's analog input (0–1023) to a PWM output range (0–255) for motor speed control.

```
// C++ code
//
int sensorValue = 0;

int outputValue = 0;

void setup()
{
  pinMode(A0, INPUT);
  pinMode(9, OUTPUT);
  Serial.begin(9600);
}

void loop()
{
  // read the value from the sensor
  sensorValue = analogRead(A0);
  outputValue = map(sensorValue, 0, 1023, 0, 255);
  analogWrite(9, outputValue);
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("     output = ");
  Serial.println(outputValue);
  delay(100); // Wait for 100 millisecond(s)
}
```

Figure 1: Original Code

To eliminate the `map()` function, since the analog input ranges from 0 to 1023 and PWM output ranges from 0 to 255, dividing the input by 4 achieves the desired scaling.

```
void loop()
{
  // read the value from the sensor
  sensorValue = analogRead(A0);

  outputValue = sensorValue / 4 ;

  analogWrite(9, outputValue);
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("     output = ");
  Serial.println(outputValue);
  delay(100); // Wait for 100 millisecond(s)
}
```

Figure 2: Modified code with simple instructions

2 Direct access to registers

We want to use registers instead of Arduino functions to set pin 11 as output, generate PWM using Timer2 and control motor speed via `OCR2A`. Pin 11 on Arduino Uno corresponds to Port B, bit 3 (PB3). We set that bit in the DDRB register writing `DDRB |= (1<<3)`.

Pin 11 on most Arduino Uno boards (ATmega328P) is OC2A, associated with Timer2 (8-bit). To generate PWM on pin 11, we'll configure Timer2 in Fast PWM mode and write directly to the `OCR2A` register. So, to use Fast PWM mode with 8-bit Timer2 (like for PWM on pin 11), we set `TCCR2A` and `TCCR2B` as in the figure below.

```
int sensorValue = 0;
int outputValue = 0;

void setup()
{
  DDRB |= (1 << PB3);  // Pin 11 output

  TCCR2A |= (1 << 7) | (1 << 1) | (1 << 0);
  TCCR2A &= ~(1 << 6);

  TCCR2B |= (1 << 1);
  TCCR2B &= ~((1 << 3) | (1 << 2) | (1 << 0));

  Serial.begin(9600);
}

void loop()
{
  sensorValue = analogRead(A0);
  outputValue = sensorValue / 4;

  OCR2A = outputValue;

  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("     output = ");
  Serial.println(outputValue);

  delay(100);
}
```

Figure 3: Modified code with direct access to registers

3 Motor speed and Temperature

We connected the TMP36 sensor to the analog input pin A0 on the Arduino. The `analogRead()` function will convert the voltage into a value between 0 and 1023. We can then map this value to control the motor speed using PWM.

```
void loop()
{
  sensorValue = analogRead(A0);

  float voltage = sensorValue * (5.0 / 1023.0);
  float temperatureC = (voltage - 0.5) * 100.0;

  outputValue = (temperatureC / 40.0) * 255;

  OCR2A = outputValue;

  Serial.print("Temp(C) = ");
  Serial.print(temperatureC);
  Serial.print("     Voltage = ");
  Serial.print(  voltage);
  Serial.print("     PWM output = ");
  Serial.println(outputValue);

  delay(100);
}
```

Figure 4: Modified code to read TMP36

Putting the slider on the TMP36 to 0°C and 40°C, we obtain respectively, 0.50 V and 0.90 V.

To keep the same motor speeds over that temperature range, we modified the code as follows:

```
void loop()
{
  sensorValue = analogRead(A0);

  float voltage = sensorValue * (5.0 / 1023.0);
  float temperatureC = (voltage - 0.5) * 100.0;

  if (temperatureC < 0) temperatureC = 0;
  if (temperatureC > 40) temperatureC = 40;

  outputValue = (temperatureC / 40.0) * 255;

  OCR2A = outputValue;

  Serial.print("Temp(C) = ");
  Serial.print(temperatureC);
  Serial.print("     Voltage = ");
  Serial.print(  voltage);
  Serial.print("     PWM output = ");
  Serial.println(outputValue);

  delay(100);
}
```

Figure 5: Affine mapping: temperature 0°C to 40°C for output 0 to 255

4 Slide switch and RGB LED

Let's now add a slide switch to control behavior (ON/OFF mode) and a common-cathode RGB LED, connecting red to pin 5 and blue to pin 6. We'll use one digital pin (pin 4) to read the switch state. If switch is ON, PWM drives motor and LED shows temperature:

- Cool (≤ 20°C) $\longrightarrow$ Blue LED ON
- Hot (≤ 35°C) $\longrightarrow$ Red LED ON

If switch is OFF, motor is OFF, LED is OFF.

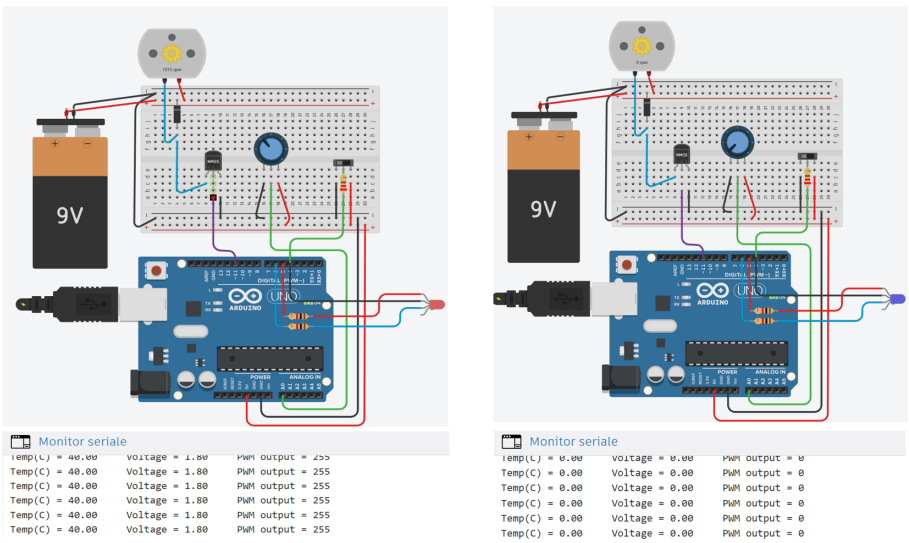

Figure 6: Final Scheme, RED and BLUE lights function of temperature

We put back a potentiometer set in order to show the temperature changes because for some reasons there was a glitch on tinkerCAD that didn't allow us to use TMP36 anymore. As we can see from the figure, when the temperature is high, the motor is at its max speed and the led is red. Contrary, when the temperature is 0 °C (or under 20 °C in general), the led is blue and rpm are lower.

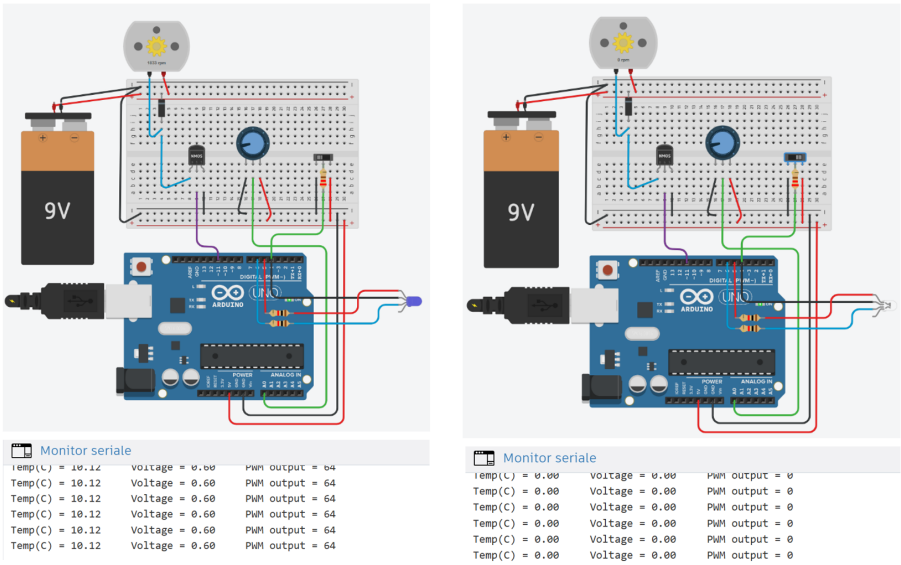

Figure 7: Blue light for $0 < T \leq 20$ °C (left image) and switch OFF (right image)

```
int sensorValue = 0;
int outputValue = 0;

void setup()
{

  DDRB |= (1 << PB3);

  // Set pins 5 and 6 as outputs for RGB LED
  pinMode(5, OUTPUT); // Red
  pinMode(6, OUTPUT); // Blue

  TCCR2A |= (1 << 7) | (1 << 1) | (1 << 0);
  TCCR2A &= ~(1 << 6);

  TCCR2B |= (1 << 1);
  TCCR2B &= ~((1 << 3) | (1 << 2) | (1 << 0));

  Serial.begin(9600);
}

void loop()
{
  // Switch is ON when pulled LOW (connected to GND)
  bool switchOn = digitalRead(4) == LOW;

  if (switchOn)
  {
    sensorValue = analogRead(A0);
    float voltage = sensorValue * (5.0 / 1023.0);
    float temperatureC = (voltage - 0.5) * 100.0;

    if (temperatureC < 0) temperatureC = 0;
    if (temperatureC > 40) temperatureC = 40;

    outputValue = (temperatureC / 40.0) * 255;
    OCR2A = outputValue;

    // LED color feedback
    if (temperatureC <= 20) {
      digitalWrite(6, HIGH); // Blue ON
      digitalWrite(5, LOW);  // Red OFF
    } else if (temperatureC >= 35) {
      digitalWrite(5, HIGH); // Red ON
      digitalWrite(6, LOW);  // Blue OFF
    } else {
      digitalWrite(5, LOW);  // Red OFF
      digitalWrite(6, LOW);  // Blue OFF
    }

    Serial.print("Temp(C) = ");
    Serial.print(temperatureC);
    Serial.print("     Voltage = ");
    Serial.print(voltage);
    Serial.print("     PWM output = ");
    Serial.println(outputValue);
  }
  else
  {
    // Switch OFF: shut down motor and LEDs
    OCR2A = 0;
    digitalWrite(5, LOW);
    digitalWrite(6, LOW);
  }

  delay(100);
}
```

Figure 8: Code Snippet for Switch and RGB LED reading

```
int sensorValue = 0;
int outputValue = 0;

void setup()
{

  DDRB |= (1 << PB3);

  DDRD |= (1 << PD5) | (1 << PD6);

  DDRD &= ~(1 << PD4);

  TCCR2A |= (1 << 7) | (1 << 1) | (1 << 0);
  TCCR2A &= ~(1 << 6);

  TCCR2B |= (1 << 1);
  TCCR2B &= ~((1 << 3) | (1 << 2) | (1 << 0));

  Serial.begin(9600);
}

void loop()
{
  // Check if slide switch (pin 4) is LOW (ON)
  bool switchOn = !(PIND & (1 << PD4));  // LOW = active

  if (switchOn)
  {
    sensorValue = analogRead(A0);
    float voltage = sensorValue * (5.0 / 1023.0);
    float temperatureC = (voltage - 0.5) * 100.0;

    if (temperatureC < 0) temperatureC = 0;
    if (temperatureC > 40) temperatureC = 40;

    outputValue = (temperatureC / 40.0) * 255;
    OCR2A = outputValue;

    // LED control using direct port access
    if (temperatureC <= 20) {
      PORTD |= (1 << PD6);   // Blue ON
      PORTD &= ~(1 << PD5);  // Red OFF
    } else if (temperatureC >= 35) {
      PORTD |= (1 << PD5);   // Red ON
      PORTD &= ~(1 << PD6);  // Blue OFF
    } else {
      PORTD &= ~(1 << PD5);  // Both OFF
      PORTD &= ~(1 << PD6);
    }

    Serial.print("Temp(C) = ");
    Serial.print(temperatureC);
    Serial.print("     Voltage = ");
    Serial.print(voltage);
    Serial.print("     PWM output = ");
    Serial.println(outputValue);
  }
  else
  {
    // Switch is OFF: turn off motor and LEDs
    OCR2A = 0;
    PORTD &= ~(1 << PD5);  // Red OFF
    PORTD &= ~(1 << PD6);  // Blue OFF
  }

  delay(100);
}
```

Figure 9: Modified code to have direct access to registers instructions